RAFAY BALOCH

CRC Press
Taylor & Francis Group

WEB HACKING ARSENAL

A PRACTICAL GUIDE TO MODERN WEB PENTESTING

Web Hacking Arsenal

In the digital age, where web applications form the crux of our interconnected existence, ***Web Hacking Arsenal: A Practical Guide to Modern Web Pentesting*** emerges as an essential guide to mastering the art and science of web application pentesting. This book, penned by an expert in the field, ventures beyond traditional approaches, offering a unique blend of real-world penetration testing insights and comprehensive research. It's designed to bridge the critical knowledge gaps in cybersecurity, equipping readers with both theoretical understanding and practical skills. What sets this book apart is its focus on real-life challenges encountered in the field, moving beyond simulated scenarios to provide insights into real-world scenarios.

The core of *Web Hacking Arsenal* is its ability to adapt to the evolving nature of web security threats. It prepares the reader not just for the challenges of today but also for the unforeseen complexities of the future. This proactive approach ensures the book's relevance over time, empowering readers to stay ahead in the ever-changing cybersecurity landscape.

Key Features

- In-depth exploration of web application penetration testing, based on real-world scenarios and extensive field experience.
- Comprehensive coverage of contemporary and emerging web security threats, with strategies adaptable to future challenges.
- A perfect blend of theory and practice, including case studies and practical examples from actual penetration testing.
- Strategic insights for gaining an upper hand in the competitive world of bug bounty programs.
- Detailed analysis of up-to-date vulnerability testing techniques, setting it apart from existing literature in the field.

This book is more than a guide; it's a foundational tool that empowers readers at any stage of their journey. Whether you're just starting or looking to elevate your existing skills, this book lays a solid groundwork. Then it builds upon it, leaving you not only with substantial knowledge but also with a skillset primed for advancement. It's an essential read for anyone looking to make their mark in the ever-evolving world of web application security.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Web Hacking Arsenal

A Practical Guide to Modern Web Pentesting

Rafay Baloch

CRC Press
Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

Designed cover image: Rafay Baloch

First edition published 2025
by CRC Press
2385 NW Executive Center Drive, Suite 320, Boca Raton FL 33431

and by CRC Press
4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

CRC Press is an imprint of Taylor & Francis Group, LLC

© 2025 Rafay Baloch

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access *www.copyright.com* or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978–750–8400. For works that are not available on CCC please contact *mpkbookspermissions@tandf.co.uk*

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data
Names: Baloch, Rafay, author.
Title: Web hacking arsenal : a practical guide to modern web pentesting / Rafay Baloch.
Description: First edition. | Boca Raton : CRC Press, 2025. | Includes bibliographical references and index.
Identifiers: LCCN 2024007038 (print) | LCCN 2024007039 (ebook) | ISBN 9781032447179 (hbk) | ISBN 9781032447193 (pbk) | ISBN 9781003373568 (ebk)
Subjects: LCSH: Penetration testing (Computer security)
Classification: LCC QA76.9.A25 B3563 2025 (print) | LCC QA76.9.A25 (ebook) | DDC 005.8—dc23/eng/20240304
LC record available at https://lccn.loc.gov/2024007038
LC ebook record available at https://lccn.loc.gov/2024007039

ISBN: 978-1-032-44717-9 (hbk)
ISBN: 978-1-032-44719-3 (pbk)
ISBN: 978-1-003-37356-8 (ebk)

DOI: 10.1201/9781003373568

Typeset in Sabon
by Apex CoVantage, LLC

Contents

Foreword xxi
Preface xxv
Acknowledgments xxvii
About the Author xxix

1 Introduction to Web and Browser 1

1.1 Introduction 1
1.2 Introduction to HTTP 1
1.2.1 Properties of HTTP 2
1.2.2 HTTP Communications 2
1.2.3 HTTP Response Codes 4
1.2.4 HTTP Request Methods 5
1.3 Common Vulnerabilities in HTTP Headers 6
1.3.1 User-Agent-Based Spoofing 6
1.3.2 Host Header Injection 6
1.3.3 Cross-Domain Referer Leakage 6
1.4 HTTP 2 7
1.5 Evolution of Modern Web Applications 7
1.5.1 Shift in Architecture 7
1.5.2 Evolution in Technology Stacks 8
1.5.3 LAMP Stack 8
1.5.4 MEAN/MERN Stack 8
1.5.5 Single-Page Applications (SPAs) 8
1.5.6 Use of Cloud Components 9
1.5.7 Serverless Architecture 9
1.6 Understanding Data Encoding 9
1.6.1 URL Encoding 11

1.6.2 Double Encoding 11
1.6.3 HTML Encoding 12
1.6.4 Base64 Encoding 13
1.6.5 Unicode Encoding 14
1.7 Introduction to Browsers 14
1.7.1 User Interface 15
1.7.2 Browser Engine 15
1.7.3 Rendering Engine 16
1.7.4 Networking 16
1.7.5 UI Backend 16
1.7.6 JavaScript Interpreter 16
1.7.7 Data Storage 16
1.8 Core Browser Security Policies and Mechanisms 16
1.8.1 Same-Origin Policy 17
1.8.2 Content Security Policy 19
1.8.3 HTTP Cookies 19
1.8.4 Iframe Sandbox 25
1.8.5 Subresource Integrity Check 25
1.8.6 HTTP Strict Transport Layer Security (HSTS) 26
1.9 Policy Exceptions versus Policy Bypasses 27
1.9.1 SOP Bypass Types 27
1.9.2 SOP Bypass—CVE-2007–0981 27
1.9.3 SOP Bypass—CVE-2011–3246 29
1.10 Site Isolation 29
1.11 Address Bar Spoofing Bugs 30
1.11.1 Address Bar Spoofing—Example 1 31
1.11.2 Address Bar Spoofing—Example 2 31
1.11.3 Bypassing Anti-Phishing Filters Using Spoofing 33
1.12 Extra Mile 35

2 Intelligence Gathering and Enumeration 37

2.1 Introduction 37
2.1.1 Enumerating ASN and IP Blocks 38
2.1.2 Reverse IP Lookup 40
2.2 Reverse IP Lookup with Multi-Threadings 41
2.2.1 Scanning for Open Ports/Services 42
2.3 Scanning Open Ports with Masscan 42

2.4 Detecting HTTP Services by Running Httpx 43
2.4.1 Scanning for Service Versions 43
2.5 Subdomain Enumeration 44
2.5.1 Active Subdomain Enumeration 45
2.6 DNSValidator 45
2.7 ShuffleDNS 46
2.8 Subbrute 47
2.9 Gobuster 48
2.9.1 Subdomain Enumeration Subdomains From Content Security Policy 48
2.9.2 Subdomain Enumeration Using Favicon Hashes 49
2.10 Putting It All Together 51
2.10.1 Passive Enumeration of Subdomains 52
2.10.2 Active + Passive Subdomain Enumeration Using Amass 57
2.10.3 Data Consolidation 61
2.11 Subdomain Takeover 62
2.11.1 Automated Subdomain Takeover Using Subjack 64
2.12 Fingerprint Web Applications 64
2.12.1 Directory Fuzzing 65
2.12.2 Discovering Endpoints Using Passive Enumeration Techniques 66
2.12.3 Enumerating Input Parameters 73
2.13 Mapping the Attack Surface Using Crawling/Spidering 75
2.13.1 Crawling Using Gospider 75
2.14 Automatic Mapping of New Attack Surface 78
2.15 Fingerprinting Web Applications 80
2.15.1 Inspecting HTTP Response Headers 81
2.15.2 Forcing Errors for Exposing Versions 81
2.15.3 Fingerprinting Using WhatWeb/Wappalyzer 81
2.15.4 Wappalyzer Browser Extensions 82
2.16 Detecting Known Vulnerabilities and Exploits 83
2.17 Vulnerability Scanning Using Nuclei 84
2.18 Cloud Enumeration 85
2.18.1 AWS S3 Buckets Enumeration 85
2.18.2 Exploiting Misconfigured AWS S3 Buckets 89
2.18.3 Exploiting Authenticated Users Group Misconfiguration 90
2.19 Extra mile 92

3 Introduction to Server-Side Injection Attacks 93

3.1 Introduction to Server-Side Injection Attacks 93
3.2 Introduction to SQL Injection 93
3.2.1 Classification of SQL Injection 94
3.2.2 SQL Injection Techniques 94
3.2.3 SQLi Data Extraction Using UNION-Based Technique 97
3.3 SQLMap Tip 1 107
3.3.1 SQL Injection to RCE 107
3.4 Retrieving Working Directory 109
3.4.1 Error-Based SQL Injection 110
3.4.2 Boolean SQL Injection 113
3.5 SQLMap Tip 2 117
3.5.1 Time-Based SQL Injection 117
3.5.2 SQLMap Tip 122
3.5.3 Second-Order SQL Injection 122
3.6 SQLMap Tip 3 129
3.6.1 Using Tamper Scripts in SQLMap 129
3.7 Remote Command Execution 133
3.7.1 RCE in Node.js 133
3.7.2 RCE in Flask Application 135
3.8 Server-Side Template Injections (SSTI) 137
3.8.1 Introduction About Templating Engines 137
3.8.2 Identifying Template Injections 139
3.9 Exploiting Template Injections 140
3.9.1 Example # 1 (Python, Jinja2) 141
3.9.2 Example # 2 (Python, Mako) 144
3.10 NoSQL Injection Vulnerabilities 146
3.10.1 MongoDB NoSQL Injection Exploitation 147
3.10.2 NoSQL Injection Real-World Examples 150
3.11 Extra Mile 151

4 Client-Side Injection Attacks 152

4.1 Introduction to XSS 152
4.2 Types of XSS 153
4.3 Reflected XSS 153
4.4 Understanding Context in XSS 154
4.5 XSS Polyglots 156
4.6 Bypassing HTMLSpecialChars 156

4.7 HTMLSpecialChars without Enquotes 157
4.8 Bypassing HTMLSpecialChars with Enquotes 158
4.9 Bypassing HTMLSpecialChars in SVG Context 159
4.10 Stored XSS 160
4.10.1 DOM-Based XSS 162
4.11 Sources and Sinks 163
4.12 Root Cause Analysis 166
4.13 JQuery DOM XSS 168
4.14 JQuery Example #1 169
4.15 JQuery Example #2 169
4.15.1 Client-Side Template Injections 170
4.16 XSS in AngularJS 171
4.17 XSS in ReactJS 173
4.18 XSS via File Upload 173
4.19 XSS Through SVG File 174
4.20 XSS Through MetaData 175
4.20.1 Weaponizing XSS 176
4.21 XSS to Account Takeover 176
4.22 XSS-Based Phishing Attack 178
4.23 XSS Keylogging 180
4.24 Content Security Policy (CSP) Bypass 180
4.25 CSP Bypass: Example #1 Unsafe Inline 181
4.26 CSP Bypass: Example #2—Third-Party Endpoints and "Unsafe-Eval" 182
4.27 CSP Bypass: Example #3—Data URI Allowed 183
4.28 CSP Bypass: Example #4—XSS Through JavaScript File Upload 184
4.29 Exploiting Browser Bugs for XSS 187
4.30 SOP and Document.Domain 187
4.31 DOM Clobbering 189
4.32 ID and Name Attribute 189
4.33 Example 1: Using Anchor Tag to Overwrite Global Variable 190
4.34 Example 2: Breaking Filters with DOM Clobbering 192
4.35 Cookie Property Overriding 193
4.36 Breaking Github Gist Using DOM Clobbering 193
4.37 Mutation-Based XSS (mXSS) 194
4.38 MXSS Mozilla Bleach Clean Function CVE 2020–6802 197

4.39 Behavior of Browser's HTML Parser 198
4.40 Extra Mile 198

5 Cross-Site Request Forgery Attacks 200

5.1 Introduction to CSRF Vulnerabilities 200
5.1.1 How Does CSRF Work? 200
5.1.2 Constructing CSRF Payload 202
5.1.3 CSRF Payloads without User Interaction 204
5.1.4 Exploiting CSRF Payload in GET Requests 205
5.1.5 CSRF Payload Delivery 206
5.2 Exploiting JSON-Based CSRF 206
5.2.1 Scenario 1: Missing Content-Type Validation and JSON Formatting 206
5.3 Scenario 2: Content-Type Is Not Validated, But JSON Syntax Is Verified 208
5.4 Scenario 3: When Server Is Expecting Application/JSON Content-Type Header 208
5.5 Automating CSRF POC Generation 208
5.5.1 OWASP ZAP POC Generator 209
5.5.2 CSRF POC Generator 209
5.6 Exploiting Multi-Staged CSRF 210
5.7 Exploiting Weak Anti-CSRF Defenses 214
5.7.1 CSRF Defenses—Weak/Predictable Anti-CSRF Tokens 214
5.7.2 CSRF Bypass—Unverified CSRF Tokens 215
5.7.3 CSRF Bypass—Referer/Origin Check 216
5.7.4 Scenario 1: Application Not Properly Validating Referer Header 217
5.7.5 Scenario 2: Weak Regex for Referer/Origin Validation 218
5.7.6 Scenario 3: Subdomain-Based Referer Validation Bypass 218
5.8 Scenario 4: Inconsistent Handling of Referer Headers 219
5.8.1 Circumventing CSRF Defenses via XSS 219
5.9 SameSite Cookies 222
5.9.1 SameSite Strict Bypass 223
5.9.2 SameSite Strict Bypass via Subdomains 224
5.9.3 SameSite Lax 225
5.9.4 SameSite Lax Bypass 225
5.9.5 SameSite None 226

5.10 Extra Mile 226

6 Webapp File System Attack 227

6.1 Introduction 227
6.2 Directory Traversal Attacks 227
6.3 Directory Traversal on Node.js App 229
6.4 Fuzzing Internal Files with FFUF 231
6.4.1 Directory Traversal and Arbitrary File Creation Vulnerability 232
6.5 File Inclusion Vulnerabilities 233
6.5.1 Local File Inclusion to Remote Code Execution 235
6.5.2 LFI to RCE via Apache Log Files 235
6.5.3 LFI to RCE via SSH Auth Log 237
6.5.4 LFI to RCE Using PHP Wrappers and Protocols 238
6.5.5 LFI to RCE via Race Condition 239
6.6 Local File Disclosure 242
6.7 File Upload Attacks 245
6.7.1 PHP Disable Functions 246
6.8 Bypassing File Upload Restrictions 249
6.8.1 Bypassing Client-Side Validation 249
6.8.2 Bypassing Blacklist-Based Filters 250
6.8.3 Apache. htaccess Override 252
6.8.4 MIME-Type Verification Bypass 253
6.8.5 Bypassing Magic Bytes 255
6.8.6 Method 1: Injecting through EXIF Data 255
6.8.7 Method 2: Raw Insertion 257
6.8.8 Vulnerabilities in Image-Parsing Libraries 257
6.9 Extra Mile 259

7 Authentication, Authorization, and SSO Attacks 260

7.1 Introduction 260
7.2 Attacks against Authentication 261
7.2.1 Username Enumeration 261
7.2.2 Username Enumeration through Timing Attack 262
7.2.3 Brute Force and Dictionary Attacks 263
7.2.4 Brute Forcing HTTP Basic Authentication 264
7.2.5 Attacking Form-Based Authentication 264
7.3 Attacking Account Lockout Policy 267
7.4 Bypassing Rate-Limiting Mechanism 268

7.4.1 Other Ways to Bypass Rate Limiting 269
7.5 Bypassing CAPTCHA 270
7.5.1 Replay Attack 271
7.6 Dynamic CAPTCHA Generation Bypass Using OCR 276
7.7 Abusing Forgot Password Functionality 279
7.7.1 Predictable Reset Token 279
7.8 Password Reset Link Poisoning via Host Header Injection 282
7.9 Attacking Authorization 284
7.9.1 Lack of Access Control 285
7.9.2 Insecure Direct Object References (IDOR) 287
7.9.3 Web Parameter Tampering 289
7.9.4 Attacking JWT 292
7.10 None Algorithm 297
7.11 Attacking OAuth 2.0 299
7.11.1 OAuth Scenario 1: Stealing OAuth Tokens via Redirect_uri 301
7.11.2 OAuth Scenario 2: Stealing Users' OAuth Tokens via Bypassing Redirect_uri 304
7.12 Attacking SAML 305
7.12.1 SAML Workflow 306
7.12.2 SAML Scenario 1: Response Tampering 306
7.12.3 SAML Scenario 2: Signature Exclusion Attack 309
7.13 Attacking Multi-Factor Authentication 310
7.13.1 Multi-Factor Authentication Bypasses 311
7.13.2 MFA Bypass Scenario: OTP Bypass 311
7.14 Web Cache Deception 314
7.15 Extra Mile 315

8 Business Logic Flaws 316

8.1 Introduction 316
8.2 Business Logic Flaws 316
8.2.1 Unlimited Wallet Balance Manipulation 317
8.2.2 Transaction Duplication Vulnerability 319
8.2.3 Improper Validation Rule Resulting in Business Logic Flaw 320

8.2.4 Exploiting Top-Up Feature to Steal Customer Balance 321
8.2.5 Lack of Validation Leads to Unlimited Card Limit 322
8.2.6 Unauthorized Manipulation of Cart Items Pre-/Post-Authentication 323
8.2.7 Loan Amount Restriction Bypass 325
8.2.8 Abuse of Feature Leads to Unlimited Wallet Balance 326
8.3 Race Condition Vulnerabilities 327
8.3.1 Race Condition Leading to Manipulation of Votes 328
8.3.2 Creating Multiple Accounts with the Same Details Using Race Condition 331
8.3.3 Exploiting Race Condition in Coupon Code Feature for Duplicate Discounts 332
8.4 Extra Mile 333

9 Exploring XXE, SSRF, and Request Smuggling Techniques 335

9.1 Introduction to XML 335
9.2 XML Structure 336
9.2.1 XML DTD 336
9.2.2 External DTD 337
9.2.3 XML Entities 338
9.3 XXE (XML External Entity) 339
9.3.1 XXE Local File Read 340
9.3.2 Remote Code Execution Using XXE 344
9.3.3 XXE JSON to XML 345
9.3.4 XXE Through File Parsing 346
9.3.5 Reading Local Files via php:// 348
9.4 Blind XXE Exploitation Using Out-of-Band (OOB) Channels 349
9.4.1 Parameter Entities 349
9.4.2 OOB XXE via HTTP 350
9.4.3 XXE OOB Using FTP 352
9.4.4 Error-Based Blind XXE 353
9.5 Server-Side Request Forgery (SSRF) 353
9.5.1 SSRF Port Scan 354

9.5.2 File Read with SSRF 356
9.5.3 SSRF in PHP Thumb Application 357
9.5.4 Validation of the Vulnerability 358
9.5.5 SSRF to Remote Code Execution (RCE) 359
9.5.6 Scanning for Open Ports 359
9.5.7 Interacting with Redis and the Gopher Protocol 361
9.5.8 Chaining SSRF with Redis for File Write to Obtain RCE 362
9.5.9 DNS Rebinding in SSRF Attacks 363
9.6 HTTP Request Smuggling/HTTP Desync Attacks 366
9.6.1 CL.TE Technique Leading to Persistent XSS 367
9.6.2 CVE-2019–20372: HTTP Request Smuggling via Error Pages in NGINX 370
9.7 Extra Mile 372

10 Attacking Serialization 373

10.1 Introduction to Serialization 373
10.1.1 Concept of Gadget 374
10.2 Insecure Deserialization/PHP Object Injection 374
10.2.1 PHP Magic Functions 376
10.2.2 PHP Object Injection—Example 376
10.2.3 PHP Object Injection in SugarCRM 378
10.2.4 Input Parameters 379
10.2.5 Finding a Magic Function 380
10.3 Insecure Deserialization—DOT NET 383
10.3.1 Deserialization of the Base64-Encoded Payload 386
10.3.2 ASP.NET Viewstate Insecure Deserialization 386
10.3.3 MAC Validation and Encryption 387
10.3.4 Exploiting with YSOSerial 388
10.3.5 Blacklist3r 388
10.4 Decoding VIEWSTATE 388
10.5 Insecure Deserialization—Python 389
10.5.1 Serializing the Data with Pickle.Dumps 390
10.5.2 Deserializing the Bytes with Pickle.Loads 390
10.6 Insecure Deserialization—Java 395
10.6.1 Gadgets Libraries in Java 396
10.6.2 Insecure Deserialization—Example 396
10.6.3 Vulnerable Code 397
10.6.4 Verifying the Vulnerability 397
10.6.5 Generating the URLDNS Payload 397

10.6.6 Obtaining RCE Using Insecure Deserialization 398
10.6.7 Blackbox Review of Java-Based Applications 401
10.6.8 Java Framework and Libraries Indicators 402
10.7 Extra Mile 402

11 Pentesting Web Services and Cloud Services 403

11.1 Introduction 403
11.1.1 Differences between RPC and REST 404
11.1.2 Monolithic versus Distributed Architecture 404
11.2 Introduction to SOAP 405
11.2.1 Interacting with SOAP Services 406
11.2.2 Invoking Hidden Methods in SOAP 406
11.2.3 SOAP Account-Takeover Vulnerability 409
11.2.4 Remote Code Execution (RCE) in SOAP Service 411
11.2.5 Finding Writable Directory 413
11.2.6 Uploading Shell to Achieve RCE 413
11.3 JSON-RPC Vulnerabilities 414
11.4 REST API 416
11.4.1 Request Methods 417
11.4.2 Identifying REST API Endpoints 417
11.4.3 Example 1: Excessive Data Exposure 418
11.4.4 Example 2: Sensitive Data Exposure 419
11.4.5 Example 3: Unauthorized Modification Using Users' Profile 420
11.5 GraphQL Vulnerabilities 420
11.5.1 Enumerating GraphQL Endpoint 422
11.5.2 GraphQL Introspection 422
11.6 Response 425
11.6.1 Information Disclosure: GraphQL Field Suggestions 426
11.6.2 GraphQL Introspection Query for Mutation 427
11.7 Response 430
11.8 Response 431
11.9 Serverless Applications Vulnerabilities 431
11.9.1 Functions as a Service (FaaS) 432
11.10 Sensitive Information Exposure 433
11.10.1 Serverless Event Injection 434
11.10.2 Analysis of Vulnerable Code 435
11.11 Extra Mile 437

12 Attacking HTML5 438

12.1 Introduction 438
12.2 Cross-Origin Resource Sharing 438
12.2.1 Weak Access Control Using Origin Header 440
12.2.2 CORS Leading to DOM XSS Vulnerability 441
12.2.3 Exploiting OpenRedirects 443
12.3 Web Storage: An Overview 443
12.3.1 Session Storage 443
12.3.2 Local Storage 444
12.3.3 Session/Local Storage API 444
12.3.4 Security Concerns with Web Storage in HTML5 445
12.3.5 Session Hijacking 445
12.3.6 Second-Order DOM XSS Using Local Storage 445
12.4 IndexedDB Vulnerabilities 447
12.4.1 Scenario—A Notes Application 448
12.5 Web Messaging Attacks Scenarios 451
12.5.1 Sender's Window 451
12.5.2 Receiver's Window 452
12.5.3 Security Concerns 452
12.5.4 Not Validating Origin in PostMessage API 452
12.5.5 DOM XSS in PostMessage API 453
12.6 WebWorkers Vulnerabilities 456
12.6.1 Interacting with WebWorker 456
12.6.2 WebWorker DOM XSS 457
12.6.3 Distributed Denial of Service Attacks Using WebWorkers 458
12.6.4 Distributed Password Cracking Using WebWorker 460
12.7 WebSockets 461
12.7.1 WebSocket DOM XSS 462
12.7.2 Cross-Site WebSocket Hijacking (CSWH) 463
12.7.3 WebSocket and Unencrypted Connections 466
12.8 UI Redressing Attacks 466
12.9 Extra Mile 471

13 Evading Web Application Firewalls (WAFs) 472

13.1 Introduction to WAF 472
13.1.1 WAF Detection Methods 472

13.1.2 Regular Expressions 473
13.1.3 Bayesian Analysis 473
13.1.4 Machine Learning 473
13.1.5 Understanding WAF Security Models: Whitelisting and Blacklisting 473
13.1.6 Whitelisting-Based Models 473
13.1.7 Blacklisting-Based Models 474
13.1.8 Fingerprinting WAF 475
13.1.9 Cookie Values 476
13.1.10 Citrix Netscaler 476
13.1.11 F5 Big IP ASM 476
13.1.12 Barracuda WAF 477
13.1.13 HTTP Response Codes 477
13.1.14 ModSecurity 477
13.1.15 Sucuri WAF 478
13.1.16 CloudFlare WAF 478
13.1.17 Connection Close 479
13.2 Bypass WAF—Methodology Exemplified at XSS 480
13.2.1 Injecting Harmless HTML 480
13.2.2 Considerations 480
13.2.3 Injecting Script Tag 480
13.2.4 Testing with Attributes and Corresponding Tags 481
13.2.5 Testing with src Attribute 481
13.2.6 Testing with Srcdoc Attribute 482
13.2.7 Testing with Action Attribute 482
13.3 Testing with Formaction Attribute 482
13.3.1 Testing with Data Attribute 483
13.3.2 Testing with href Attribute 483
13.3.3 Testing with Pseudo-Protocols 484
13.3.4 Using HTML Character Entities for Evasion 487
13.3.5 Injecting Event Handlers 488
13.3.6 Injecting a Fictitious Event Handler 489
13.3.7 Injecting Lesser-Known Event Handlers 489
13.3.8 Injecting Location Object 490
13.3.9 Bypass Using Unicode Separators 491
13.3.10 Using SVG-Based Vectors 493
13.3.11 Bypassing WAF's Blocking Parenthesis 493
13.3.12 Bypassing Keyword-Based Filters 493
13.3.13 Character Escapes 494
13.3.14 Constructing Strings in JavaScript 494
13.3.15 Accessing Properties through Syntactic Notation 495

13.3.16 Bypassing Keyword-Based Filters Using Non-Alphanumeric JS 496
13.3.17 Alternative Execution Sinks 496
13.3.18 Bypassing WAF's Decoding Entities 498
13.3.19 Case Study: Laravel XSS Filter Bypass 498
13.3.20 Bypassing Recursive Filters through Tag Nesting 500
13.3.21 Bypassing Filters with Case Sensitivity 500
13.3.22 Bypassing Improper Input Escaping 501
13.3.23 Bypassing Using DOM XSS 503
13.3.24 Example for Disallowed Keywords 504
13.3.25 Using Window.Name Property 504
13.4 Setting the Name Property 505
13.5 Example 1: Using the Iframe Tag 505
13.6 Example 2: Window.open Function 505
13.7 Example 3: Anchor Tag 506
13.7.1 Bypassing Blacklisted "Location" Keyword 506
13.7.2 Variations Using Different Browser Properties 507
13.7.3 Bypassing WAF Using HPP 507
13.8 Example with XSS 507
13.9 Example with SQL Injection 508
13.10 Extra Mile 508

14 Report Writing 509

14.1 Introduction 509
14.2 Reporting Audience 509
14.3 Executive Summary 510
14.3.1 Structure of an Executive Summary 510
14.3.2 Executive Summary Fail 512
14.3.3 Recommendations Report 513
14.4 Findings Summary 513
14.4.1 Overall Strengths 514
14.4.2 Overall Weaknesses 515
14.5 Historical Comparison 515
14.6 Narrative of the Report 516
14.7 Risk Assessment 516
14.7.1 CVSS Scoring 517
14.7.2 Limitations of CVSS 519
14.8 Risk Matrix 519
14.8.1 Risk Assessment and Reporting 520

14.9 Methodology 520
14.10 Technical Report 520
14.11 Organizing the Report 524
14.12 Report Writing Tools 525
14.12.1 ChatGPT for Report Writing 525
14.12.2 Prompt 1 525
14.12.3 Prompt 2 526
14.12.4 Prompt 3 527
14.12.5 Prompt 4 528
14.13 Report Writing Tips 529
14.14 Extra Mile 530

Index 531

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Foreword

Well, congratulations, dear reader! You hold in your hands nothing less than a delightful tome of arcane secrets, occult knowledge, and powerful spells for our modern age. I mean this in the most sincere way possible. Rafay Baloch is nothing less than an eldritch wizard with powers far beyond mere mortals, and I've been aware of this mad sorcerer for the better part of a decade.

Back in 2014, both Android and Apple iPhones were beating the pants off of RiM BlackBerry and Windows Phones in terms of smartphone market share. The Android mobile operating system enjoyed a comfortable lead of about ten points ahead of Apple's mobile OS, then called iOS, and pretty much everyone agreed this was because Android phones were cheaper to build and cheaper to buy.

Of course, smartphones were in the midst of taking over the world. Even back then, your phone was becoming what futurists and trans-humanists call an "exocortex", a device that exists outside your own brain but complements it with memory and calculation powers beyond the reach of ordinary humans. We are literally evolving to use these devices to store and recall all our most important, intimate details of our subjective lives. It's an understatement to say that security is pretty important for mobile devices.

Anyway, back to 2014: At this time, I was working at Rapid7, running a community vulnerability disclosure program, aimed primarily at folks who wrote Metasploit modules for publicly known vulnerabilities. Nearly every exploit module in Metasploit exploits vulnerabilities known as "*n*-days", or bugs that were already known and nominally fixed by software producers. This sort of exploit is distinct from "zero-days", or bugs that are novel, new, and have no patch available. These Metasploit modules are useful for things like testing defenses, validating patches, or finding those neglected systems that for whatever reason haven't been patched yet. But, every once in a while, a researcher would come up with a 0-day, and sought out my help with disclosing these findings to software vendors so patches could happen.

So, along came Rafay, bouncing into my email inbox with what I first assumed would be a fairly normal vulnerability disclosure affecting some down-market Android smartphones. Cool, but ultimately, pretty normal.

Bugs happen, and Android, being a complex software project and available on a wide variety of platforms, was bound to have some interesting bugs, especially in older versions. Also, since Google was the major power behind Android, we all assumed they'd be just as responsive Android bugs as they were with their flagship product, Google Chrome. We expected an easy discover-report-patch cycle.

Well, what happened next kind of blew my mind. It turned out that Rafay had discovered some pretty serious bugs in Android's stock web browser, WebView (this was before Android standardized on Chromium). On top of that, they were apparently hard to fix. In fact, they were so hard to fix that security gatekeepers at Google invented a policy, possibly on the spot, that they wouldn't be fixing the "unsupported" Android operating system version codenamed "Jelly Bean", also known as Android 4.3. This policy was never actually published anywhere, and seemed to discount the fact that at the time, the vast majority of Android phones in the world—about 70% or so—were actively running version 4.3 or earlier. In real numbers, this accounted for just shy of **a billion devices**.

Again, these are devices that people rely on practically as a second brain. When you lose control of that, it means you lose control of the details of your entire life. What became clear to me while working with Rafay was that, first, we (as a species) absolutely needed to be paying more attention to core Android vulnerabilities, and by extension, the whole world of web client and server vulnerabilities in a mobile context. Second, Rafay was one of the very few people in the world capable and willing to discover, then articulate and publish, these sorts of software vulnerabilities.

Both government and corporate entities around the world employ people like Rafay to probe the technologies we all rely on, like mobile devices, web browsers, and web applications for software weaknesses and vulnerabilities. I guarantee you that they are sitting on secret exploits, right now. Unlike Rafay, though, those people are generally sworn to secrecy, and who knows what they're using this private, secret knowledge for. Lucky for us, there are a few people out there with Rafay's particular set of skills who are willing to go public with their findings. We rely on the efforts of public security researchers, and Rafay Baloch in particular, to advance our collective understanding of how these technologies actually work and are actually implemented. We can then use this knowledge to make them more resistant to the predations of spies and criminals. This is core hacker culture at work: information wants to be free, and dangerous information tends to get defanged by public disclosure.

It's been my pleasure to know and work with Rafay over the years to help people be safe and secure in their computing lives—which is really their living lives. I hope that, if you're reading this book, you will take a page or two out of Rafay's life's work and make the effort to disclose your next

vulnerability to the technology company or open source project in a coordinated, and ultimately public, way. Sharing our learnings is really the best way, and maybe the only way, we can get better at defending our privacy, our security, and our safety in a hyperconnected and distributed world.

—Tod Beardsley, Shmethical Hacker
Huge Success LLC, PacketFu Security

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Preface

Today, more than at any other time in history we are now inexorably linked to a cyber-world. Our lives revolve around technology and multiple web applications. The vast majority of people have no idea of how they work and do not really care. They only want to know that they are safe and secure. However, because of this growing trend, the need for robust web security is now more essential than ever.

My journey through cyber security, specifically in web application penetration testing, has been driven by a need, a desire for knowledge, and a commitment to advance the digital defenses that protect our personal and professional lives.

This book captures the extensive research and firsthand experience gained in real-world penetration testing and bug bounty programs. It attempts to fill in some critical knowledge gaps and will hopefully serve as a practical guide from someone that has actual experience in the field.

One of the notable aspects of this book is its attention to the dynamic nature of web security threats and the recognition of the constant evolution of digital threats, strategies, and techniques, which are not only effective today but also adaptable for future challenges. This guarantees the book's relevance and will help prepare readers to anticipate and counter emerging threats.

This book takes a balanced approach to web security, blending solid theoretical foundations with a practical orientation. It offers a well-rounded view which moves beyond just theoretical discussions, incorporating real-world scenarios and insights from actual penetration testing experiences, and provides detailed case studies and practical examples, helping to bridge the gap between theory and practice.

Because this book caters to a wider audience, from beginners in web security to seasoned professionals looking to update their skills, and everyone in between, and covers a wide range of topics, from fundamental concepts to advanced techniques, it will become not only a valuable resource but also the go-to reference guide for everyone in this line of work.

As you delve deeper and deeper, you are invited to explore the intricate and vital world of web security. My hope is that at the end of this journey you will have gained not only substantial knowledge but also practical skills and a strategic approach, which will help to prepare you to navigate and address the challenges and opportunities of our digital world.

Acknowledgments

The author is highly indebted to Tod Beardsley, Jonathan Sharrock, Etizaz Mohsin, Farhan Memon, Alex Infuhr, Soroush Dallili, File Descriptor, Prakhar Prasad, Dr. Asim Ali Rizvi, Dr. Erum Ranjha, Muhammad Ahmed, and Faisal Bukhari for their insights and feedback, which were crucial in enhancing the quality and depth of this work.

SPECIAL ACKNOWLEDGMENTS

Muhammad Samaak: Samaak's significant contributions include implementing, testing, verifying, and troubleshooting the source codes and scenarios presented in the book. His invaluable assistance and innovative suggestions regarding topic coverage have been pivotal to the book's completion.

Kamran Khan: Kamran has assisted in developing various scenarios for this book, specifically those related to client-side penetration testing. His work includes helping authors in developing test scenarios, verifying test case scenarios, and assessing their technical correctness.

Hammad Shamsi: Hammad has played a pivotal role in motivating the writing and completion of this book. He has contributed by assembling the missing pieces and assessing the technical correctness across various chapters.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

About the Author

Rafay Baloch is a globally renowned cybersecurity expert and white-hat hacker with a proven record of identifying critical zero-day security vulnerabilities in numerous web applications, products, and browsers. His discoveries have been instrumental in safeguarding the privacy and security of millions of users worldwide. Baloch has received various accolades, including being named one of the "Top 5 Ethical Hackers of 2014" by Checkmarx, one of the "15 Most Successful Ethical Hackers Worldwide", and one of the "Top 25 Threat Seekers" by SC Magazine. In addition, Reflectiz listed him among the "Top 21 Cybersecurity Experts You Must Follow on Twitter in 2021".

On March 23, 2022, the Inter-Services Public Relations (ISPR) recognized Baloch's significant contributions to the field of cybersecurity with the Pride of Pakistan award. Baloch is also the author of "Ethical Hacking and Penetration Testing Guide", published by Taylor & Francis in 2014.

Rafay has presented his research at various international cybersecurity conferences, including Black Hat, Hack In Paris, HEXCON, the 10th Information Security Conference in Greece, the CSAW Conference, and many others. He is frequently sought after for his insights and analysis on current cybersecurity topics, appearing in national and international mainstream media outlets such as *Forbes*, *WSJ*, *Independent UK*, *BBC*, *Express Tribune*, *DAWN*, and many others.

Baloch has also served as senior consultant for cyber security at the Pakistan Telecommunication Authority (PTA), the national telecom regulator.

Rafay Baloch is the founder of REDSECLABS, a company specializing in cybersecurity consulting, training, and a variety of other cyber security–related services at the global level. The book features several sample codes and "extra mile" exercises designed to enhance learning. To apply these concepts practically, we encourage you to visit our website at www.redseclabs.com. On the site, you'll find blog posts that explore these exercises and other resources mentioned throughout the book, along with showcases of our research work.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Chapter 1

Introduction to Web and Browser

1.1 INTRODUCTION

Web applications have become an integral part of the modern digital landscape. Over the past decade, they have evolved remarkably in terms of technology, features, and functionality, aimed at creating a rich user experience. However, each advancement in functionality has brought its own set of complexities. Additionally, browsers competing for market dominance constantly introduce unique features to web applications, with many implementing security policies and mechanisms in different manners. Due to the absence of a consistent reference implementation, the implementation of security policies has been browser-specific and highly diverse. Such variations not only expand the threat surface but also create opportunities for attackers to exploit these inconsistencies.

The intersection of web applications with web browser technologies is a critical area of focus. This chapter will highlight how browser-specific features and security implementations relate to their importance in the broader context of web security. We will also dive into the world of browser security, exploring core security policies and mechanisms introduced by browsers to protect web applications. Understanding these fundamentals is extremely vital to web security.

1.2 INTRODUCTION TO HTTP

Hypertext Transfer Protocol (HTTP) is the protocol that runs the World Wide Web. At a fundamental level, it is based on a client–server architecture, whereby the client requests content (typically via browsers) and the server/application delivers the response. The request from the client to servers can be routed via intermediary devices such as reverse proxies, load balancers, and web application firewalls (WAFs). The default port for transmitting HTTP is TCP (Transmission Control Protocol) port 80, but it can also operate over different ports and can be encapsulated within other protocols.

DOI: 10.1201/9781003373568-1

1.2.1 Properties of HTTP

Following are some of the core properties of HTTP:

Statelessness: HTTP is a stateless protocol, which means that two requests do not have any relation to each other. However, to manage states such as login, mechanisms such as cookies are used.

Lack of Inherent Encryption: HTTP is an unencrypted protocol, which means that any intermediary devices on the network, such as routers or proxies, will be able to read and modify the traffic. To solve this, HTTPS encapsulates HTTP within a TLS/SSL (transport layer security/secure sockets layer) encryption layer.

Extensibility: HTTP is designed to be extensible, which means that it uses headers in both requests and responses to convey meta data and other relevant information. This allows for the implementation of new features without making modifications to the core.

Reliability: HTTP is transported over TCP, which means that the reliability is guaranteed. TCP is a connection-oriented protocol, which ensures that data packets are delivered in order and without any errors.

1.2.2 HTTP Communications

HTTP communications are based upon HTTP request and HTTP response. Client sends HTTP request to the server asking for a certain resource, and the server responds with the HTTP response. Let's analyze a sample HTTP request.

The following HTTP request attempts to access the "index.html" file hosted at **redseclabs.com/index.html**.

HTTP Request

```
GET /index.html HTTP/1.1
Host: www.redseclabs.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4430.
212 Safari/537.36
Referer: https://google.com
```

GET /index.html HTTP/1.1

GET refers to the HTTP method used for this request followed by the resource being requested. In this case, it is "index.html". This is followed by the version of HTTP protocol, that is, HTTP/1.0.

Host: www.redseclabs.com

The host field refers to the host to which the request is being submitted. Host field is required as one IP can host multiple websites or virtual hosts.

User-Agent: Mozilla/5.0

The user-agent field indicates the browser and operating system being used to access a website. It is commonly utilized to deliver custom pages, especially for ensuring cross-compatibility with different browsers. The term **Mozilla/5.0** in the user-agent string is a historical artifact; most browsers start their user-agent string with **Mozilla/5.0** for compatibility reasons. Toward the end of the string, the presence of **Chrome/90.0.4430.212** indicates the browser version.

Referer: www.google.com

"Referer" header is used to indicate to the server the URL of the web page where the user is coming from. For example, if the **Referer** header shows **www.google.com**, it means that the user arrived at the current website by clicking a link on Google search.

This HTTP request results in the following HTTP response:

Example: HTTP Response

```
HTTP/1.1 200 OK
Date: Sun, 26 Nov 2023 12:00:00 GMT
Server: Apache/2.4.41 (Unix)
Content-Length: 450
Content-Type: text/html; charset=UTF-8
Connection: close

<html><body> <h1>Welcome to RedSecLabs!</h1></body></html>
```

Let's analyze the HTTP response line by line:

HTTP/1.1 200 OK

This field indicates that the HTTP 1.1 protocol is being used and the request has been successfully processed as indicated by the HTTP status code of 200.

Date: Sun, 26 Nov 2023 12:00:00 GMT

This field provides the timestamp when the response was sent.

Server: Apache/2.4.41 (Unix)

This field indicates that the server is running Apache 2.4.41 and is hosted on the Unix operating system. Revealing this field can potentially aid attackers, and hence it is not mandatory and can be removed or even replaced with a fictitious value.

Content-Length: 450

This field specifies the size of the response content in bytes; in this case, it is 450 bytes.

Content-Type: text/html; charset=UTF-8

This field indicates the type of content being sent (HTML) and the character encoding being used, that is, "UTF-8".

Connection: close

This field indicates that once the response has been delivered, the TCP/IP socket will be closed, requiring users to open a new one before communicating further. Alternatively, it can be set to "Keep-Alive", which will keep the connection open for subsequent requests.

1.2.3 HTTP Response Codes

HTTP response codes are represented by three digits, indicating the status of the response. Each status code represents different response categories:

Table 1.1 Common HTTP response codes

Code Range	*Category*	*Description*
1xx	**Informational**	Used to indicate a server is changing protocols, such as HTTP to WebSocket.
2xx	**Success**	Indicating that the request has been understood correctly.
3xx	**Redirection**	Signifying that further action is required to complete the request, such as redirecting to a new URL.
4xx	**Client error**	Indicates that the request contains incorrect or inaccurate information.
5xx	**Server error**	Indicates that the request has encountered an issue processing the request.

1.2.4 HTTP Request Methods

HTTP includes a variety of methods, but the most essential ones are **GET** and **POST**. While these methods are commonly used in web interactions, other methods are optional and may serve specific purposes. **GET** is traditionally used to retrieve content, and **POST** is used to submit content to the server. However, **GET** can also be used to send content to the server. Let's take an example of a login form using a **GET** request to process username and password to authenticate the user.

Request

```
GET /login.php?username=myusername&password=mypassword
HTTP/1.1
Host: www.redseclabs.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.
4430.212 Safari/537.36
Content-Type: application/x-www-form-urlencoded
Content-Length: 34
Connection: close
```

There are few problems with this approach from a security standpoint:

- GET requests are logged in server logs, hence anyone with access to these logs, such as unauthorized users exploiting security vulnerabilities, could see the complete URL. This becomes a concern when endpoints inadvertently leak logs or when there's unauthorized access to the logs beyond intended administrative oversight.
- Browsers and intermediary proxies may cache **GET** requests.
- Users may bookmark such URLs containing sensitive information and when sharing them would inadvertently could potentially expose sensitive data.

Now, let's see how the same request with same parameters would look when processing via **POST** request:

Request

```
POST /login.php HTTP/1.1
Host: www.redseclabs.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.
4430.212 Safari/537.36
```

```
Content-Type: application/x-www-form-urlencoded
Content-Length: 34
Connection: close
```

`username=myusername&password=mypassword`

The username and password are not part of the URL and are part of the HTTP request body and hence are not vulnerable to aforementioned security issues.

1.3 COMMON VULNERABILITIES IN HTTP HEADERS

Let's talk about some of the common vulnerabilities that could arise through misconfiguration. We will dive into these vulnerabilities at lengths in their respective sections in this book.

1.3.1 User-Agent-Based Spoofing

User-agent value can be manipulated and hence cannot be trusted by servers. However, yet many administrators tend to implement rate limiting and other security mechanisms on the basis of the user agent.

1.3.2 Host Header Injection

In case if the server does not validate the host header, it might be possible for an attacker to inject a malicious host value. This could lead to attacks such as web cache poisoning, password reset, and redirecting users to malicious websites.

1.3.3 Cross-Domain Referer Leakage

Cross-domain Referer leakage occurs when the referrer URL contains sensitive information such as session ID, tokens, and password and when the user navigates to a different origin. For example, in the following request, the Referer header contains session token that is leaked to free image uploader domain. An attacker having control of this website can potentially use this to hijack victims' sessions and take over accounts.

Websites can include "**Referer-Policy** either as a meta tag or as an HTTP header to suggest when to include a Referer header when navigating to a different website. Here are some configurations:

no-referrer: Never send the Referer header.
same-origin: Send the Referer header only for same-origin requests.
strict-origin-when-cross-origin: Send only the origin for cross-origin requests.

1.4 HTTP 2

The most recent upgrade to HTTP 1.1 is HTTP 2. It provides significant upgrades in terms of speed and performance. Several key enhancements allow HTTP 2 to work more efficiently, such as Multiplexing, which allows multiple resources to be delivered concurrently over a single connection. Due to these improvements, websites no longer need to split the content across multiple domains. A single connection can handle multiple requests, thus effectively reducing the latency.

For example, in HTTP 1.1, when a user wants to watch a video on YouTube or on another video-streaming platform, the process involves loading various page elements such as the video file, scripts, CSS files, and JavaScript. This loading is managed through the use of multiple TCP connections. However, browsers typically limit the number of concurrent connections to a single domain, which can create bottlenecks, especially on pages with many resources.

HTTP 2 comes with another feature, "server push", which allows the server to send content to the client by predicting what the client will need. On the contrary, server push can be abused by the server by potentially pushing malicious resources to the client or spoofing existing objects. Similarly, in HTTP 2, attackers can send large header frames with excessive header field sizes. Servers may allocate memory based on header sizes, leading to denial of service.

1.5 EVOLUTION OF MODERN WEB APPLICATIONS

During the past decade or so, the web has undergone a major shift in terms of the technology stack, architecture, and infrastructure. The use of web services and RESTful API (application programming interface) has become widespread, facilitating integration between heterogeneous services. This evolution reflects a broader industry shift toward scalability, efficiency, and reliability. Here is a summary of these developments.

1.5.1 Shift in Architecture

In the past, many applications were built as monolithic structures, meaning that all the components were tightly coupled and interdependent. This meant that a failure in one component could fail the entire application, creating a single point of failure. Recently, there has been a major shift toward microservices architecture. This is achieved by breaking down applications into smaller, independently functioning units. We will discuss microservices and security issues at length in Chapter 11.

1.5.2 Evolution in Technology Stacks

The evolution in the technology stacks represents a shift in the way web applications are developed and deployed. Newer technologies and architectures such as microservices and serverless computing and newer database technologies such as NoSQL have emerged, offering alternatives to the traditional LAMP stack. Let's understand different stacks.

1.5.3 LAMP Stack

At the time of writing this book, PHP is utilized by 76.6% of all websites. A significant number of developers who use PHP as their server-side programming language also prefer Linux as their operating system, Apache as their HTTP server, and MySQL as their database server. Together, these technologies form what is known as the LAMP stack, an acronym for Linux, Apache, MySQL, and PHP/Perl/Python. Over the years, each of these individual components has evolved.

Linux is continuously updated and remains a preferred choice for server environments. While Apache is widely used, it faces competition from web servers such as Nginx. MySQL largely remains popular, but alternate database systems like PostgreSQL have emerged. PHP has seen significant improvements over time. Python has grown in popularity, particularly in emerging fields, and Perl has become less prominent.

1.5.4 MEAN/MERN Stack

While all individual components forming LAMP are being continuously updated, developers have been moving away from these components except for Linux, and there has been a shift toward more advanced stacks such as MEAN/MERN stacks.

MEAN (MongoDB, Express.js, Angular, Node.js) and MERN (MongoDB, Express.js, React, Node.js) are popular stacks for development. These stacks offer a unified language, that is, JavaScript, across both client and server sides, making development more efficient and streamlined. The adoption of NoSQL databases such as MongoDB, Cassandra, and Redis provides alternatives to traditional relational databases. While MERN has become the popular choice, it has brought its own set of problems concerning security, for instance, the introduction of a new class of vulnerabilities such as Node.js injection, NoSQL injection, dependency injection, and so on.

1.5.5 Single-Page Applications (SPAs)

Single-page Applications have been recently popularized and are intended to improve user experience. Unlike traditional websites, a new page is fetched from the server every time you navigate or submit a form. SPA initially loads

the entire web page and all the components just once. After the initial load, SPA dynamically updates content on the same page, eliminating the need for reloading the web page.

SPAs often rely upon RESTful APIs or Graph APIs to fetch data, making them suitable for integration with microservice architecture. They are built with JavaScript frameworks such as AngularJS, React, and many more.

Due to the heavy reliance upon JavaScript to render and dynamically update content, SPAs are often vulnerable to DOM-based cross-site scripting. We will explore this Chapter 4.

1.5.6 Use of Cloud Components

Cloud computing has significantly facilitated the development of microservices by enhancing scalability. Technologies such as Docker and Kubernetes are crucial for managing microservices and providing containerization for effective segregation and scaling.

Similarly, in web development practices, the DevOps culture has become prominent, focusing on collaboration and automation between development and operations teams. Complementing this, CI/CD pipelines automate the software delivery process, enabling faster and more efficient releases in a containerized environment.

1.5.7 Serverless Architecture

Serverless architecture has become increasingly popular in the context of SPAs and the deployment of microservices. Despite its name, serverless architecture does involve servers. In this model, the cloud provider is responsible for managing the servers; developers write code and only pay for the time code is executing. Serverless architecture comes with its own set of security challenges. We will delve into it in the web services chapter (Chapter 11).

1.6 UNDERSTANDING DATA ENCODING

Encoding in web applications is used to ensure that the communications follow a specified set of rules and standards. URLs can only contain a limited set of characters, which are mainly alphanumeric (letters and numbers) and certain special characters. When an input is inserted outside this allowed set, these characters need to be encoded to prevent ambiguity.

The process of encoding and decoding generally happens behind the scenes: users can input various characters, and browsers and applications take care of the encoding and decoding. When a user supplies a disallowed character into the browser, it is automatically encoded before processing the request.

Classification	Included characters	Encoding required?
Safe characters	Alphanumerics [0–9a–zA–Z] and unreserved characters. Also reserved characters when used for their reserved purposes (e.g., question mark used to denote a query string)	**NO**
Unreserved characters	– . _ ~ (does not include blank space)	**NO**
Reserved characters	: / ? # [ ] @ ! $ & ' ( ) * + , ; = (does not include blank space)	**YES** [1]
Unsafe characters	Includes the blank/empty space and " < > % { } | \ ^ `	**YES**
ASCII Control characters	Includes the ISO-8859-1 (ISO-Latin) character ranges 00-1F hex (0-31 decimal) and 7F (127 decimal)	**YES**
Non-ASCII characters	Includes the entire "top half" of the ISO-Latin set 80-FF hex (128-255 decimal)	**YES**
All other characters	Any character(s) not mentioned above should be percent-encoded.	**YES**

Figure 1.1 Table representing characters that require encoding.

Figure 1.1 [**https://perishablepress.com/stop-using-unsafe-characters-in-urls/**] represents a chart that explains the characters that can be treated as "Safe" and the ones that should be encoded.

It is worth noting that reserved characters need encoding only when they are used beyond their defined purpose. Let's discuss the main types of data encodings being used in web applications:

- URL encoding
- HTML encoding
- Base 64 encoding
- Unicode encoding

1.6.1 URL Encoding

URL encoding, also known as percent encoding, is a process used to encode reserved characters in a URL. In URL encoding, characters not part of the allowed set for URLs are replaced with a percentage symbol (%) followed by their hexadecimal value. For example, the ampersand character ("&") is commonly used as a separator in a query string, it must be encoded to prevent ambiguities. For example, consider the following URL:

Example

```
https://example.com/login.php?username=tmgm&password=t&mgm
```

In this example, the password contains ampersand "&", which if not encoded will be treated as a parameter and will set a password to "t" and "mgm". To prevent this, ampersand "&" is encoded as "%26", resulting in the final URL:

Example

```
https://example.com/login.php?username=tmgm&password=t
%26mgm
```

Here is Table 1.2 representing common characters that are encoded and their encoded versions:

1.6.2 Double Encoding

In double encoding, the characters are encoded twice: the first level of encoding converts the character into a percent-encoded form. The second level of encoding is applied to the percent-encoded characters. In other words, this

Table 1.2 Common characters encoded and encoded versions

Character	*Encoded Version*
Space	%20
Double Quote ("')	%22
Less Than (<)	%3C
Greater Than (>)	%3E
Pound (#)	%23
Ampersand (&)	%26
Slash (/)	%2F
Plus (+)	%2B
Equal (=)	%3D

means that it will apply to the percent sign (%) itself, along with the encoded character hexadecimal value being encoded another time.

For example, if you wanted to double encode the character "<", you would first encode it as %3C. Then, you would encode the "%" as %25, resulting in %253C as the double-encoded form.

This is a common technique that can be used to bypass WAFs and application-level filters that decode the URLs once. If a WAF decodes only once, it would see **%253C** as harmless, not realizing that the second decoding would turn it into "<" character. We will dive into these techniques later.

1.6.3 HTML Encoding

In HTML, certain characters have special meanings and can lead to ambiguities if not handled correctly. For instance, characters "**<**" and "**>**" can represent the opening and closing of an HTML tag. To ensure that these characters are displayed in textual format, rather than being interpreted as HTML syntax, they have to be HTML encoded.

HTML encoding involves replacing these special characters with character entities. For example, a less-than sign "**<**" in HTML can be replaced with "**&lt**", and for a great-than sign "**>**", we would use "**&gt**". Additionally, HTML encoding is not just limited to characters that have special meaning in HTML syntax. It can also represent characters not readily available on standard keyboards. For example, the copyright symbol "**©**" can be represented in HTML as "**&copy**".

As per HTML specification, all character references must start with an ampersand "**&**" sign, this can be followed by multiple variations such as decimal and hexadecimal encoding. Here are various ways to represent these characters.

Table 1.3 Various ways to represent these characters

Characters	*Named Entity*	*Decimal Encoding*	*Hexadecimal Encoding*

<	`<`	`<`
		`&#000060`
			`<`
			`&#0x3C`
			`&#0000x3C`

>	`>`	`>`
		`&#000062`
			`>`
			`&#0x3e`
			`&#0000x3e`

'	`'`	`'`
		`&#000039`
			`'`
			`&#0x27`
			`&#000027`

"	`"`	`"`
		`&#000034`
			`"`
			`&#0x22`
			`&#000022`

Note: From the table, you can see that we can use leading zeros in decimal and hexadecimal forms.

1.6.4 Base64 Encoding

Base64 encoding can be used to allow binary data to be represented as an ASCII string.

One of the most common uses of base64 is for transmitting email attachments safely as email servers often alter or misinterpret certain characters such as newlines, leading to ambiguities or corruption of the data.

For instance, consider the string "**Hello\nWorld**", where "**\n**" represents a newline character. The string will be treated as follows:

Example

```
Hello
World
```

This string contains a newline after the letter "**o**". In ASCII, this string is represented by the decimal values.

Example

```
72 101 108 108 111 10 119 111 114 108 100
```

In this example, the byte sequence "10" represents the "newline" character. Email systems might not interpret this character correctly. Hence, by encoding these characters using base64 encoding, we can represent them as ASCII characters, thus eliminating characters like newlines that are problematic in email systems.

The support for base64 encoding and decoding is widely available across all web programming languages. JavaScript provides the "**btoa**()" and "**atob**()" functions for handling base64.

It is worth mentioning that base64 is commonly confused by developers as an encryption scheme rather than an encoding scheme. Even in real-world engagements, you might find instances where sensitive information is encoded with base64, resulting in the exposure of sensitive data. Figure 1.2 demonstrates base64 encoding/decoding using "btoa" and "atob" functions:

```
> btoa("Hello\nWorld")
<· 'SGVsbG8KV29ybGQ='
> atob("SGVsbG8KV29ybGQ=")
<· 'Hello\nWorld'
```

Figure 1.2 Base64 encoding/decoding.

1.6.5 Unicode Encoding

Unicode contains a large number of characters from numerous languages around the world. For instance, if you wish to include Arabic or Persian text in the web page, you will find that these characters are not part of ASCII as it was primarily developed for English and has limited character sets. This is where Unicode comes into play.

To effectively map such a large set of characters, Unicode utilizes several encodings such as UTF-8, UTF-16, and UTF-32. Let's explore how Unicode can be used to represent common characters:

Table 1.4 Unicode to represent common characters

Characters	*Unicode Equivalent*
<	`\u003c` `%u003c`
>	`\u003e` `%u003e`
'	`\u0027` `%u0027`
"	`\u0022` `%u0022`

Understanding how encodings work can provide great aid in bypassing WAFs and filters that rely upon blacklists. We will build upon these concepts in Chapter 13.

1.7 INTRODUCTION TO BROWSERS

Browsers act as an interface to access web applications and are responsible for interpreting and displaying content to the end user. They are primarily responsible for rendering pages by processing HTML, CSS, and JavaScript. In the context of the ever-evolving web security landscape, web browsers have expanded their role beyond just rendering pages and continuously introducing security controls to protect the privacy and security of the users.

For example, to protect user privacy, many browsers offer built-in measures such as enhanced cookie controls, private browsing modes, blocking third-party trackers, and many others. On the security front, browsers have implemented certain built-in security policies such as the same-origin policy (SOP), which restricts how content from different origins can interact within the browser, whereas several optional security mechanisms are implemented in the form of headers and can be utilized by web administrators to enhance security.

Browsers support extensions and plug-ins that provide additional functionality, such as ad blocking, password management, and so on. While these additional functionalities can improve the security of a user, these functionalities can also be weaponized by an attacker and serve as a weakness. Discussing browser length is a complex topic and is beyond the scope of this chapter. Figure 1.3 gives a high-level overview of core browser components:

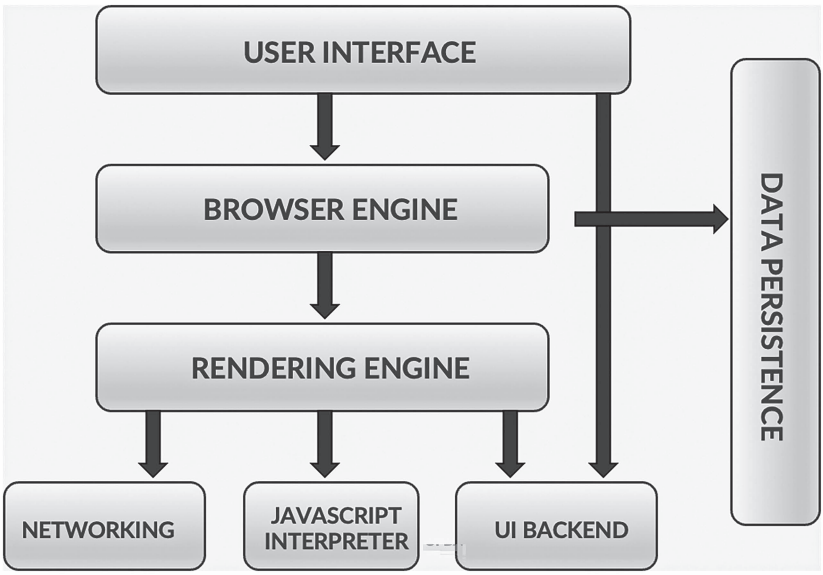

Figure 1.3 High-level overview of a browser's internals.

Let's briefly talk about each of the components.

1.7.1 User Interface

This represents the HTML and CSS and displays the parsed content on the screen. Browsers utilities include everything you see except the windows where the web page is being rendered. For example, the address bar, back/forward buttons, and bookmarking menu are all parts of the user interface.

1.7.2 Browser Engine

It acts as an interface between the UI and the rendering engine. For example, when a user interacts with the browser interface, such as typing a URL,

clicking a link, or interacting with the form, the browser engine is responsible for processing the command.

1.7.3 Rendering Engine

A rendering engine is an integral part of the web browsers, it essentially converts HTML, CSS, and JavaScript into a visual and interactive pages. Each browser employs a specific engine: Chrome and Opera use Blink, Firefox uses Gecko, and Safari operates on WebKit. These engines process the markup and scripting to create the Document Object Model (DOM) of the web page, apply styles from CSS, execute JavaScript for dynamic content, and then render the layout and visual representation of the page on the screen.

Given that each web browser utilizes a specific rendering engine, a vulnerability in an engine such as Blink, Gecko, or WebKit, could expose all browsers relying on that particular engine to potential security risks.

1.7.4 Networking

This component is responsible for making underlying network calls such as HTTP and DNS.

1.7.5 UI Backend

This is used to access underlying operating system methods such as combo boxes, user boxes, and so on.

1.7.6 JavaScript Interpreter

This component is responsible for parsing and executing JavaScript code present within web pages.

1.7.7 Data Storage

This component is responsible for storing data on the client side. This includes mechanisms such as cookies, Web Storage, IndexedDB, WebSQL, and FileSystem.

1.8 CORE BROWSER SECURITY POLICIES AND MECHANISMS

Browser vendors have introduced several security policies and mechanisms to protect their users. These policies are implemented by default or may be implemented by each browser in a different way. To safeguard its users,

browser vendors have implemented a wide array of security policies and mechanisms. These policies range from controlling the type of resource that can be loaded at a granular level to enforcing strict isolation between different websites.

1.8.1 Same-Origin Policy

The same-origin policy (SOP) is one of the most fundamental and core policies in browsers. The policy in its essence prevents web pages in one origin from being able to access properties in a different origin. Origin is normally referred to as a combination of scheme, domain, and port. In simple words, two web pages are considered to be of the same origin, if their scheme, domain, and port numbers are matched.

It is worth mentioning here that SOP is inconsistent and heterogeneous in nature and hence its implementation across browsers may vary. One example from the past of Internet Explorer consists of a scheme and a host; however, ports are not taken into consideration.

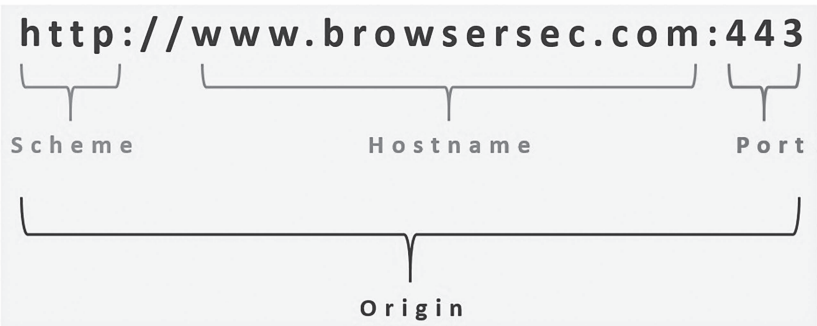

Figure 1.4 Origin in same-origin policy.

To get a better understanding of SOP, let's take an example of the following code which is hosted at output.jsbin.com, which uses Ajax request to fetch the response of gmail.com and write it to the web page using a document. write function.

Code

```
<script>
xhr = new XMLHttpRequest();
xhr.open('GET', 'www.gmail.com', true);
xhr.onreadystatechange = function () {
  if (xhr.readyState === XMLHttpRequest.DONE && xhr.
    status === 200) {
```

```
    document.write(xhr.responseText);
    }
};
xhr.send();
</script>
```

The screenshot in Figure 1.5 illustrates that access to "**www.gmail.com**" from "**https://output.jsbin.com**" has been blocked due to hostname mismatch.

The following table (Table 1.5) outlines the rules for interactions between different origins and specifies the conditions under which the origins will be treated as the same.

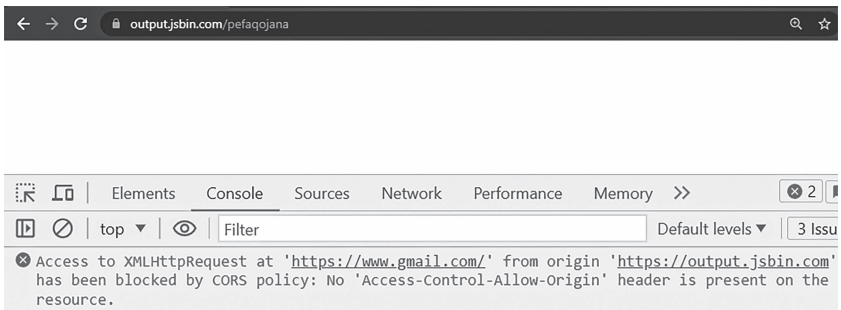

Figure 1.5 SOP violation.

Table 1.5 Rules for interactions between different origins

Origin 1	*Origin 2*	*Same Origin*
http://store.example.com/page.html	**http://store.example.com/**newpage.html	**YES**
http://**store.example.com/**page.html	http://news.example.com/page.html	**NO**
http://store.example.com:**80/**page.html	http://store.example.com:**8080/**page.html	**NO**
https://**store.example.com**:8443/page.html	**ABOUT:BLANK**	**YES**
https://store.example.com/**dir/page.html**	https://store.example.com/**dir/subdir/page.html**	**YES**

Note: In the table, store.example.com and about:blank are shown to be on the same origin. This might be confusing for some readers. "About:blank" has no origin and inherits the origin of the document that created it. For instance, if a page at https://store.example.com opens a new window to about:blank, then the about:blank page will inherit the origin of https://store.example.com.

1.8.2 Content Security Policy

Content security policy (CSP) is a security policy widely supported by all modern browsers and is designed to mitigate injection attacks such as XSS, clickjacking, and other code injection attacks resulting from unauthorized script execution. The policy is discretionary and can be implemented through "Content-Security-Policy" headers. Let's see an example of how CSP can be set to only allow browsers to include JavaScript from a specified resource, that is, "*http://code.jquery.com/jquery-1.11.0.min.js*".

Example:

```
Content-Security-Policy:script-src http://code.jquery.com/
jquery-1.11.0.min.js;
```

This policy would ensure that only JavaScript from the specified jQuery library is executed, blocking any other scripts that are not from this source. CSP was introduced as a very stringent policy, and hence, breaking multiple websites was not practical from a real-world perspective; however, as more levels were introduced, the policy became less stringent and practical.

CSP Levels progressed from Level 1 with basic directives to Level 3, having advanced controls allowing for granular management of resources. CSP defines directives to control resources a page can load, which would allow developers to specify the sources to be whitelisted. This includes scripts, styles, images, and more.

CSP Level 1—This level introduced basic directives that would allow developers to specify the sources to be whitelisted. This includes scripts, styles, images, and more.

CSP Level 2—This level offered developers more control over which origins can embed your site into frames.

CSP Level 3—This level is currently under development, and only parts of it have been implemented under modern browsers and offer more granular control of resource interactions.

We will explore more about CSP and its potential bypass due to misconfiguration in the subsequent chapters.

1.8.3 HTTP Cookies

As mentioned previously, HTTP is a stateless protocol and hence does not know about previous interactions from the same client. To overcome this limitation, HTTP cookies are used. When a client first submits a request to the web server, the server responds with a "**Set-Cookie**" header containing a cookie value. The cookie value is then stored on the client's browser. With

every subsequent request to the web server, the browser submits the cookie. This allows the server to recognize the client and maintain the session state.

A cookie is defined based on **Domain** and **Path** attributes in contrast to the Same-Origin Policy (SOP), which relies on the scheme, host, and port. For instance, consider the following cookie.

Example

```
Set-Cookie: key=anyvalue; domain=example.com; Path=/search/
```

In this example, a cookie is set for the example.com domain and is specifically scoped to /search/path. This means that the cookie containing "**key=value**" will be sent by the browser only when the requests are directed to URLs within the "**example.com**" domain and under '**/search/**" path.

Unlike SOP, cookies are not strictly bound to a single origin, provided the cookie's Domain and Path attributes allow it. This allows a website at one origin to set a cookie that can be read by another origin.

1.8.3.1 Domain-Level Cookie Scoping

As described earlier, in the context of the cookie, the scope of the cookie is set via domain and path parameters. However, the scope when loosely set can lead to discrepancies. Consider the following example:

Example

```
Set-Cookie: key=anyvalue; domain=.example.com
```

In this example, the cookie is with the domain attribute to "**.example.com**" using a leading dot, which represents an older convention used for setting cookies. When a cookie is set with a domain as "**.example.com**", it becomes accessible to all of its subdomains such as **sub.example.com** or **tmgm.sub.example.com**.

Modern browsers have standardized this behavior, and when you set **domain=example.com** (without a leading dot) or **domain=.example.com** (with the leading dot), the cookies will be accessible across both specified domains and all their subdomains.

Note: The correct way to restrict and limit scope would be not to include the domain attribute at all.

1.8.3.2 Cookie Tossing Vulnerability

As discussed before, when a cookie is not tightly scoped to the current domain, it can be accessed by subdomains of the primary domain. In case an attacker manages to gain control of any subdomains, the implications of

these vulnerabilities can be widespread, including the ability for an attacker to fixate the session token, which leads to account takeover. Here is how the attack works:

Step 1: An attacker having control of **vulnerable.example.com** sets the cookie named "**sessionID**" to a known value.

Example

Set-Cookie: sessionID=***valueknowntoattacker; domain***=*example.com*

Step 2: When a victim visits an attacker-controlled subdomain (**vulnerable.example.com**), the browser stores the value.
Step 3: The victim visits **example.com** and logs in to the application using the fixated session ID.
Step 4: When the victim logs in to the application, the application doesn't generate a new **SessionID**.
Step 5: The attacker uses the same **session ID** to take over the victim's account.

Cookie tossing is possible even when a cookie with a specific name is already set. When the browser receives two cookies with the same name, the browser will treat the request as valid and send both of them.

Example

Set-Cookie:SessionID=*setbytheserver;****SessionID***=*attacker known;* ***domain***=*.example.com*

Most of the applications will process the first parameter in case of duplicates. In that case, we can force our cookie by adding longer paths. This is because cookies with longer paths take precedence as per details documented under RFC 6265 [**https://datatracker.ietf.org/doc/html/rfc6265#section-5.4**].

```
2.  The user agent SHOULD sort the cookie-list in the following
    order:

    *  Cookies with longer paths are listed before cookies with
       shorter paths.

    *  Among cookies that have equal-length path fields, cookies with
       earlier creation-times are listed before cookies with later
       creation-times.
```

Figure 1.6 Excerpt from RFC 6265 about cookie order.

Example of shorter path

```
Set-Cookie: SessionID=setbytheserver; domain=.example.com;
path=/
```

Example of attacker setting longer path

```
Set-Cookie:SessionID=attackerknown; domain=.example.com;
path=/admin
```

When the browser processes these headers, it will store both cookies. However, when accessing a path that matches both cookies such as "/admin", the cookie with a longer path that is set by the attacker will take precedence.

1.8.3.3 Cookie Bomb Vulnerability

A cookie bomb occurs when a website sets an excessively large number of cookies or a couple of cookies that are very large in size. Browsers have limitations on the number of cookies they can store, the limit is set on the number of cookies per domain or the size of the cookies. On the server side, too many cookies can lead to excessive data being sent in HTTP headers, which can increase load times and potentially overload the server.

Let's take a look at the following JavaScript code, which will set multiple cookies starting at "**testcookie1**" and incrementing until **99**, each with value having **4000** "A":

Example

```
// Setting the domain
let baseDomain = 'hackerone.com';
// Create a string of 4000 'A's for cookie value
let cookieValue = 'A'.repeat(4000);
// Loop to set multiple cookies
for (let cookieNum = 1; cookieNum < 99; cookieNum++) {
// Setting a cookie with incremental names and the long
string as value
document.cookie='testCookie${cookieNum}=${cookieValue}
;Domain=${baseDomain}';
}
```

Upon execution of this JavaScript code in the context of hackerone.com, we can see that cookie value is set.

Consequently, accessing hackerone.com and its subdomains will lead to an error.

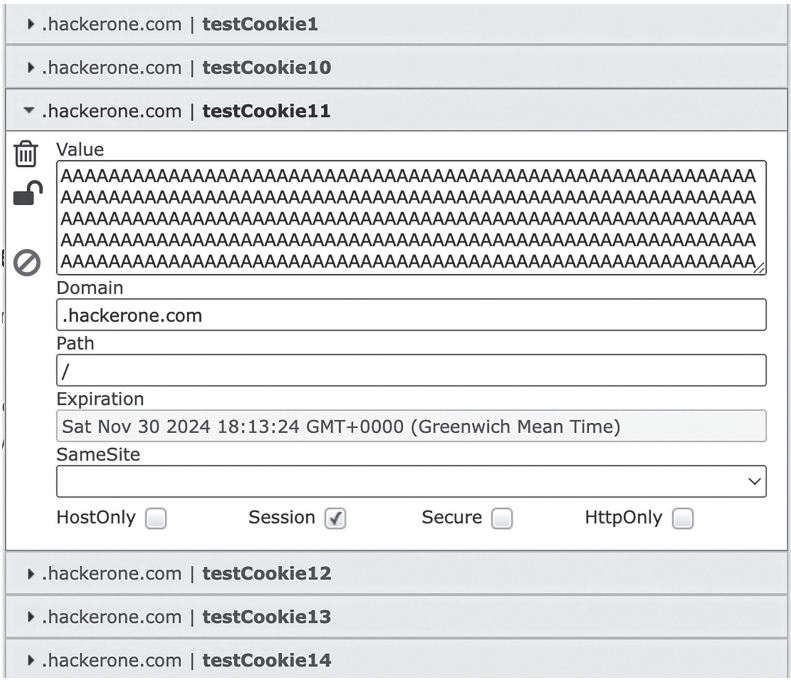

Figure 1.7 Cookies set on hackerone.com.

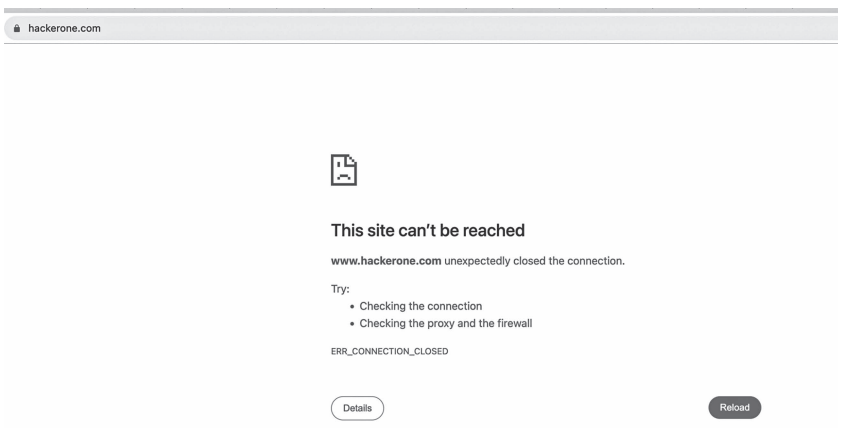

Figure 1.8 Hackerone.com inaccessible after setting large cookies.

You cannot set this cookie as cross-origin due to restrictions imposed by the Same-Origin Policy (SOP); however, a vulnerable code in an application might allow users to set cookies cross-domain through user-controllable parameters.

1.8.3.4 Session Expiry and Validation

HTTP cookie contains attributes such as "**Max-age**" and "**Expires**" attributes, which allow users to set the cookie. "**Max-age**" defines the maximum time the cookie will be valid for. For instance, if the max-age is set to "**3600**", it means that the cookie is valid for 3,600 seconds, which is equivalent to one hour. Upon expiry, the cookie will be automatically deleted from the browser.

Example

```
Set-Cookie: key=value; Max-Age=3600; Path=/path; Domain=
example.com
```

The "**Expires**" attribute defines a specific time when the cookie will expire.

Example

```
Set-Cookie: key=value; Expires=Sun, 31 Dec 2024 23:59:59 GMT;
Path=/path; domain=example.com
```

The obvious security risk is that, if a cookie expiry timeline is set to a long term, an attacker obtaining access to the cookies will be able to maintain access to it for an extended period. Several web applications have chosen to accept this risk and compensating control such as re-authenticating users when accessing sensitive functions or in case discrepancies are detected.

1.8.3.5 Cookie Protection

There are two essential cookie flags that have a drastic impact on security in terms of HTTP cookies. One of the flags is "secure", which indicates to the browser that this cookie is to be sent only in secure connections, such as a TLS connection. Another security-related flag is "httponly". This flag instructs the browser to disallow access from JavaScript. Here is an example of the implementation:

Example

```
Set-Cookie: key=value; Max-Age=3600; Path=/; domain=example.
com; HttpOnly; Secure
```

HTTP cookies can also be set to Same Site Origin, which will tell the browser when to send cookies. This is used as a protection mechanism for cross-site request forgery (CSRF), which will be discussed in the relevant Chapter 5.

1.8.4 Iframe Sandbox

The "Iframe" tag is a powerful HTML element that allows websites to embed web pages into the current document. When a page is loaded in an Iframe, it will load all the contents from the destination address including HTML, CSS, and JavaScript. This presents a security risk as the content loaded from an external web page, which may be malicious in nature, would result in the security of the parent website being compromised.

To address this issue, HTML5 specifications introduced the "**sandbox**" attribute for Iframe. This offers a granular control over the type of content that should be loaded. Following is an example of a sandbox attribute being used to load example.com.

Example

```
<iframe sandbox src="http://example.com/"></iframe>
```

The default settings of the sandbox attribute are very restrictive in nature. These include blocking JavaScript execution and disabling form submissions among other constraints. However, it also provides developers the flexibility to fine-tune the allowed content through various attributes such as allow-forms, allow-popups, allow-same-origin, and allow-scripts. Additionally, CSP includes a sandbox directive which, when implemented, applies similar restrictions across the entire document.

1.8.5 Subresource Integrity Check

Web applications often load external resources such as JavaScript libraries and CSS files. These resources are sometimes hosted on third-party servers like code.jquery.com for jQuery. However, if such a domain is compromised and its content is replaced with a malicious version, any website embedding these resources could also be compromised.

This is due to JavaScript's ability to manipulate the web page, which could result in consequences such as theft of sensitive data and become vectors for spreading malware. While CSP allows the whitelisting of domains to load resources, however, it cannot protect against compromised external resources from the whitelisted domains.

To address this issue, browsers have introduced a security feature called subresource integrity (SRI) check, which ensures that scripts have not been modified since they were first loaded. This is accomplished through the integrity attribute, which takes in input that contains the checksum or hash value (such as SHA-256, SHA-384, or SHA-512) of the external file, which is used to ensure the integrity of the loaded file.

Example

```
<script/src="https://code.jquery.com/jquery-3.6.0.min.js"
integrity="sha256-tmgm3212 . . ." crossorigin="anonymous">
</script>
```

In this example, the integrity attribute contains the hash value of the expected content of jquery-3.6.0.min.js. If the content inside this file changes, the hash value will be changed, preventing the altered script from loading.

1.8.6 HTTP Strict Transport Layer Security (HSTS)

Websites can use HTTP to HTTPS, and forced redirect can also ensure that the website is only accessible over HTTPS. However, this alone will not prevent protocol downgrade attacks. This is a form of cryptographic attack, used to downgrade an encrypted connection to a weak mode of operation, hence making it trivial for an attacker to intercept and decrypt the data.

To address this issue, browsers introduced HTTP Strict Transport Security (HSTS), which instructs browsers to convert all HTTP requests to HTTPS and hence prevent attackers from exploiting insecure HTTP connections.

HSTS policy can be set using the "Strict-Transport-Security" header in the HTTP response. Here is an example:

Example

```
Strict-Transport-Security: max-age=31536000; includeSub
Domains
```

The header uses the max-age directive, which specifies the duration (in seconds) that the browser should remember that a site should only be accessed using HTTPS. The HSTS also includes the "includeSubdomains" attribute, which means that the policy should apply to all the subdomains, not just the primary domain.

HSTS can contain a "preload" directive, which is a list that is hardcoded into browsers to always use HTTPS, even before any interaction with the website. HSTS preload list is a collaborative effort by major web browsers. Website owners have to fulfill certain criteria for inclusion in the HSTS preload list.

Example

```
Strict-Transport-Security: max-age=31536000; includeSub
Domains; preload
```

1.9 POLICY EXCEPTIONS VERSUS POLICY BYPASSES

From the prior literature in this chapter, it is evident that security policies are not stringent in nature and offer flexibility to developers and website owners. This flexibility is often facilitated through the use of policy exceptions Here, it is important to distinguish between a policy bypass and a policy exception.

A policy bypass is generally considered as a vulnerability within the browser, which involves exploiting a loophole to circumvent an effectively implemented policy, whereas, policy exceptions are defined as legitimate scenarios in which browsers allow controlled circumvention of policies.

1.9.1 SOP Bypass Types

SOP bypasses have become prevalent mainly due to the increasing complexity of document object model (DOM) and JavaScript. This is then compounded by server-side functionalities such as redirects. The majority of SOP bypasses are logical bugs, that is, which are the result of a logical confusion or mismatch between different layers and components within the browsers. For example, one component might detect a Null-Byte and stop the execution of a code, while another chooses to completely ignore it and execute the code.

Several SOP bypasses are not confined to individual browsers as they exist within shared components, potentially impacting multiple browsers. For example, a bypass occurring in the WebKit rendering engine affects all browsers built on top of the same engine. Similarly, SOP bypasses found in plugins like Java can potentially impact all browsers that support these plugins.

There are several categories of SOP bypasses; however, similar to the work of Schwenk et al. (2017), we have divided them into four different types: partial read, full read, partial write, and full read and write. For each category, potential attacks falling within their respective categories have also been mentioned.

1.9.2 SOP Bypass—CVE-2007–0981

Let's examine a classic case of an SOP bypass due to a layer mismatch, discovered in older versions of Firefox and recorded as **CVE-2007–0981**. Let's take a look at the POC:

POC

```
location.hostname = "evil.com\x00www.bing.com"
```

Table 1.6 Categories of SOP bypasses

SOP Bypass	*Description*	*Attacks Category*
Partial Read	Partial Read access in the SOP world implies reading certain properties/ sub-properties during cross-origin communication, otherwise not permitted by the SOP. There are certain exceptions where the partial read is legitimately allowed by the policy, for instance, it is permitted to read the "**width**" and "**height**" parameters of cross-origin images by design.	**Side Channel Attacks such as Cross-Origin CSS attacks, and Cross-Origin Data Leaks**
Full Read	Full Read access in the SOP world implies a script on one origin being able to read all the properties of a web page on another origin. This category also consists of exploits evading restrictions for Local File Access.	**Cross-Schema Data exposure attacks, Cross-File Attacks**
Partial Write	Partial Write in the SOP world is the ability to modify certain properties during cross-origin communication. Certain properties such as **location.href** and **location.hash** can be modified regardless of whether their window object belongs to a different origin. There are other DOM properties such as window.name that persist across origins.	**Browser Spoofing vulnerabilities**
Full Read and Write	Full Read and Write access in the SOP world implies a script on one origin being able to read and write all properties of another origin. Full Read and Write is a result of a JavaScript execution in context of another origin. We have not defined Full Write as a separate category, as Full Write in almost all cases results in UXSS (universal cross-site scripting).	**UXSS, Cross-Zone Scripting**

In this POC, the **location.hostname** property is set to **evil.com**, followed by a null byte "**\x00**" and then bing.com. Null bytes are commonly recognized as string terminators in many programming languages. However, in this case, the DOM, a part of the rendering engine, does not treat **\x00** as a string terminator. Therefore, it interprets "**evil.com\x00www.bing.com**" as a subdomain of bing.com.

On the contrary, the DNS resolver which is a part of the networking layer does recognize a null byte as a string terminator stops the execution and treats the origin as **evil.com**, disregarding the rest of the string. Consequently, it would allow an attacker to set, alter, or delete cookies for **bing.com** and its subdomains.

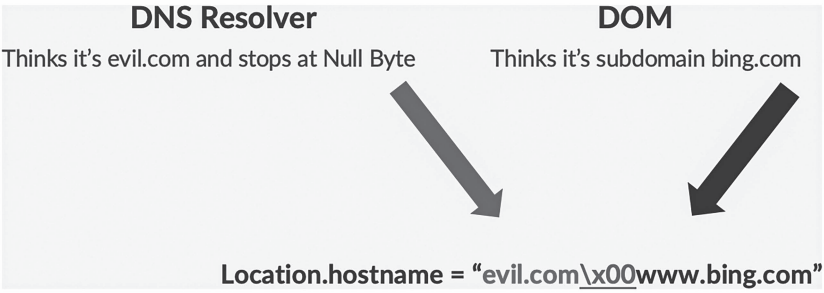

Figure 1.9 Layers mismatch between DNS resolver and DOM.

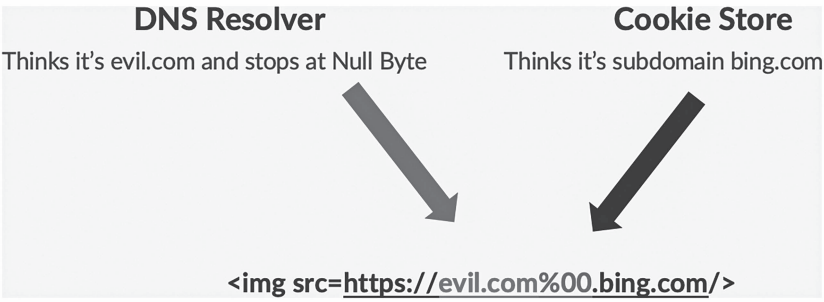

Figure 1.10 Layers mismatch between DNS resolver and cookie store.

1.9.3 SOP Bypass—CVE-2011–3246

CVE-2011–3246 is a similar example and affects older versions of Safari browser. The SOP bypass is yet another very similar case of logical confusion and layer mismatch. However, this time the mismatch is between the cookie store part of the persistence layer and the DNS resolver from the networking layer.

POC

```
<img src="https://evil.com%00.bing.com">
```

1.10 SITE ISOLATION

The majority of SOP bypasses occur due to different origins sharing the same renderer process. This is particularly common in the case of cross-site framing and pop-ups. A renderer process is responsible for handling the rendering

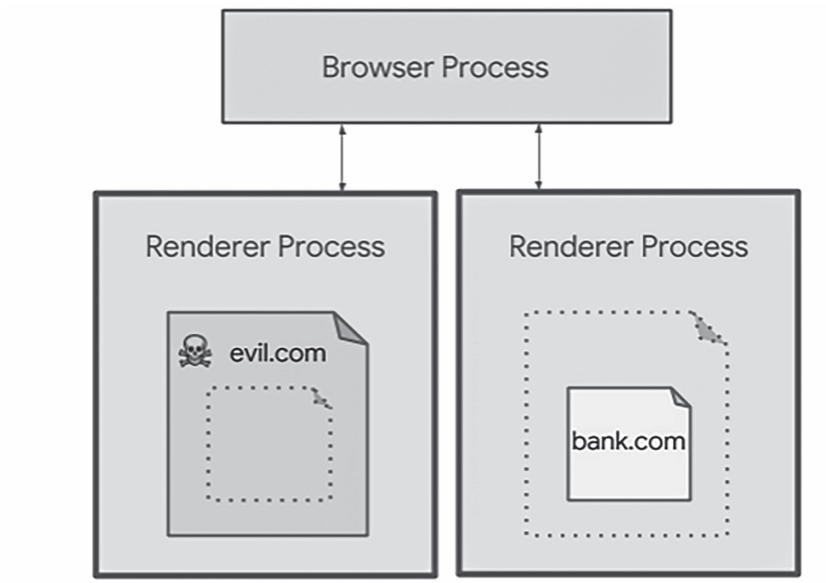

Figure 1.11 Site isolation.

Source: Ref. https://security.googleblog.com/2018/07/mitigating-spectre-with-site-isolation.html

of web pages. This includes functionalities such as parsing HTML and CSS and executing JavaScript.

Previously, browsers did not isolate different origins into separate renderer processes. Hence, when an SOP bypass was found, it allowed scripts from one origin to potentially access data from another origin within the same process.

In site isolation, each process runs in its separate renderer process segregated at an OS level. With site isolation, even if an SOP bypass occurs within the renderer process, it is much more challenging to access cross-origin data. The following diagram (Figure 1.11) demonstrates how evil.com is loading bank.com via Iframe and how under the hood a single web page is split between two renderer processes.

1.11 ADDRESS BAR SPOOFING BUGS

As per Google [**https://bughunters.google.com/about/rules/6625378258649088/google-and-alphabet-vulnerability-reward-program-vrp-rules**], address bar is the only security indicator in modern browsers that can be relied upon. This seems logical because the address bar is the primary way for users to confirm whether they are on the correct website.

Browser vendors are aware of this and therefore have implemented built-in controls to ensure that the content displayed in the browser corresponds to the URL in the address bar.

Address bar spoofing vulnerability occurs when the address bar points to the correct domain that the user is attempting to visit; however, the content of the domain is controlled by the attacker. Under the hood, browser-based spoofing vulnerabilities are executed by exploiting flaws in the browser to create the illusion that users are on the legitimate domain. Let's take a look at a couple of examples:

1.11.1 Address Bar Spoofing—Example 1

In JavaScript, several functions allow users to delay the execution of an event or to execute an event at regular intervals. These methods are powerful and can be utilized to hunt for address bar spoofing vulnerabilities. Let's look at an example of this technique:

Code

```
<script>
w = window.open("www.facebook.com", "_blank");
setTimeout(function(){w.document.write("<html>This  is
not Facebook</html>")},5000);
</script>
```

This script opens the Facebook website in a new window using the "window.open" function and then attempts to overwrite the page with the text "This is not Facebook" using document.write after a delay of five seconds. The execution is delayed using the "setTimeout" function. During this process, the browser may fail to update the address bar, so while the address bar still points to facebook.com, the content displayed is actually controlled by the attacker. Here is an example of Yandex Browser (CVE(2020–7369)) preserving the address bar using the same technique.

1.11.2 Address Bar Spoofing—Example 2

Another technique to trigger address bar spoofing is the use of a non-existent port to preserve the address bar along with the use of JavaScript timing functions to induce appropriate delays. Let's look at an example of this technique:

Code

```
function spoof(){
document.write("<h1>This is not Bing</h1>");
```

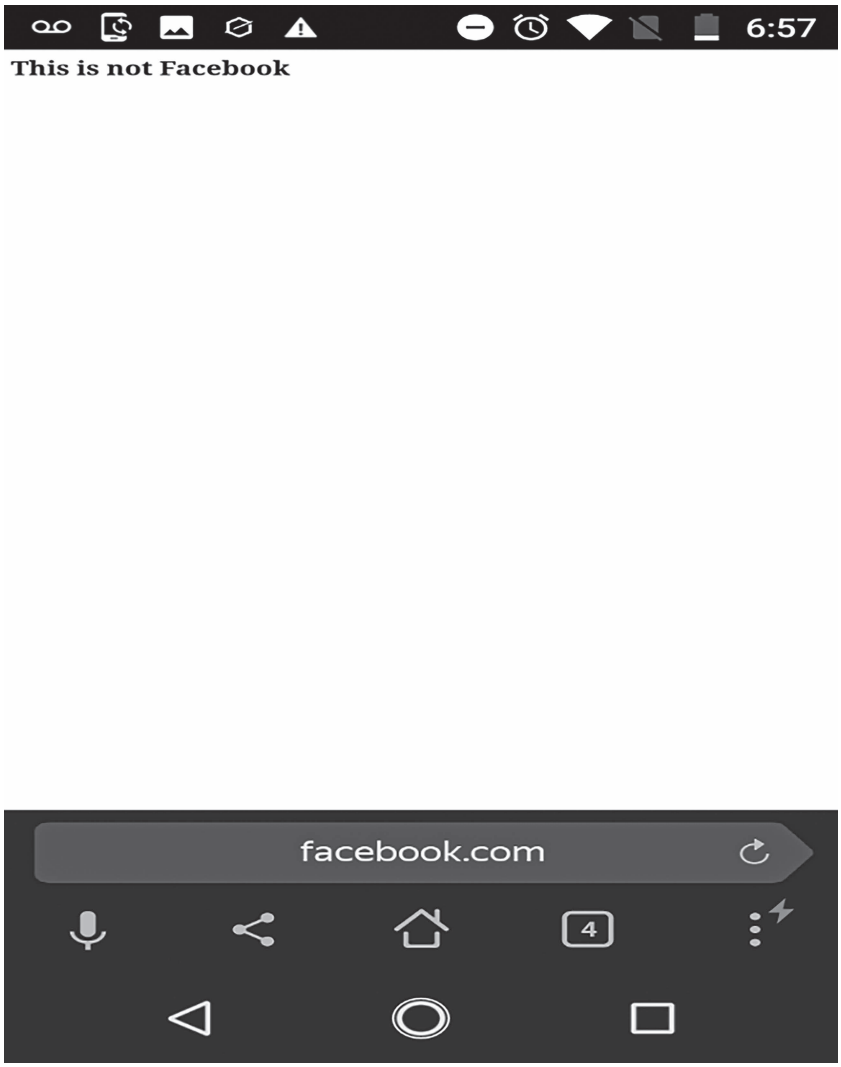

Figure 1.12 Yandex browser address bar spoofing.

```
document.location = "https://bing.com:1234";
setInterval(function(){document.location="https://
bing.com:1234";},9800);
};
```

The script uses document.location function to redirect the user to "**https://bing.com**" on a non-standard port, "**1234**". Depending upon the browser, it

will take time before eventually returning the connection timeout error, since no service is running on this port. During this timeframe, as the browser attempts to load "**https://bing.com:1234**", the setInterval function is repeatedly invoked, which causes the redirection to the URL, thus reinstating the process at defined intervals.

This process keeps the spoofed URL preserved in the address bar, while at the same time allowing attackers to modify the content of the web page.

This will be fired after every certain interval, hence preserving the URL in the process, while allowing attackers to write content to the document resulting in address bar spoofing. Ideally, the correct behavior for the browser would be to redirect to "about: blank" when the setInterval function is called for redirection.

The same POC has been used to trigger address bar spoofing vulnerabilities inside Safari browsers multiple times along with other popular browsers such as Microsoft Edge and DuckDuckGo. Interestingly, Safari browser hides the port, and hence the only URL that is visible to the user is bing.com further.

1.11.3 Bypassing Anti-Phishing Filters Using Spoofing

Anti-phishing filters that rely solely on the URL in the address bar to identify phishing pages may fail to detect such pages in the case of an address bar spoofing vulnerability. Since, these filters depend on the origin information from the address bar, they can be deceived by a spoofed URL. As a result, if the browser preserves the spoofed URL in the address bar, these anti-phishing filters would treat it as legitimate.

It is also possible to deceive anti-phishing filters even in modern browsers that are not vulnerable to the address bar spoofing vulnerability. This is because several modern browsers will update the address bar to "**about:blank**" when a user attempts to write content to a new window instead of preserving the URL, as a measure to prevent address bar spoofing vulnerabilities.

This countermeasure can, however, be used to deceive anti-phishing filters by opening a new window and writing content to an "**about:blank**" page. Consequently, when an anti-phishing filter encounters "**about:blank**", it cannot perform a site reputation check, which leads to bypass. To illustrate, consider the following code:

Code

```
function spoof() {
   var x = open('about:blank');
   base64 = 'VE1HTQo=' //phishing page encoded in base64
   x.document.body.innerHTML = atob(base64);
}
```

Figure 1.13 Address bar spoofing in Safari browser.

The code opens "about:blank" in a new window and writes a base64-encoded version of a phishing page to the web page. The base64-encoded content is decoded at the runtime and written to the web page.

1.12 EXTRA MILE

HTTP/2 Security Vulnerabilities: Explore the HTTP/2 protocol and the new kinds of security issues it introduces. The PortSwigger research team has provided comprehensive references on this topic [***https://portswigger.net/research/http2***].

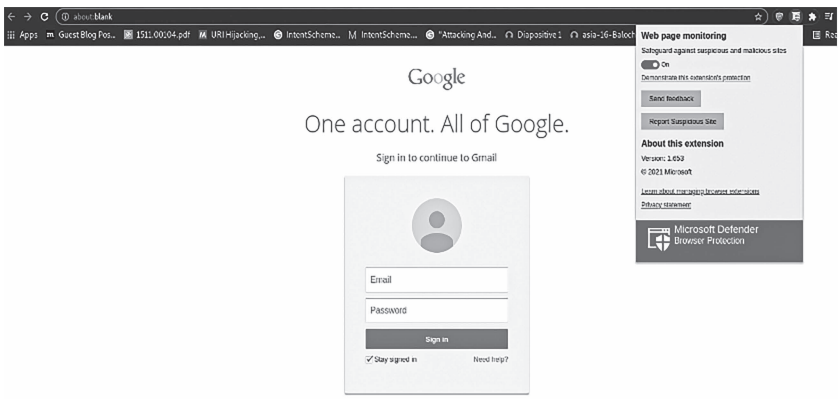

Figure 1.14 Microsoft Defender browser protection bypass.

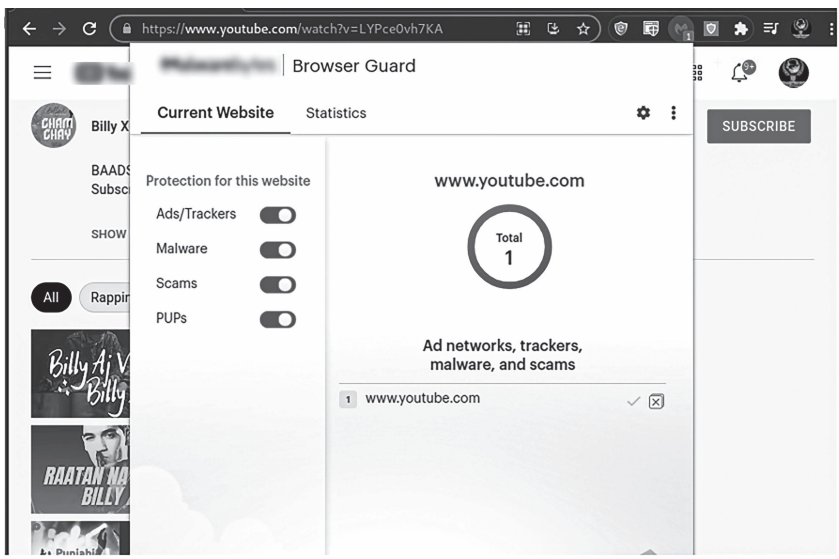

Figure 1.15 MalwareBytes Browser Guard.

SOP Exceptions: Enhance your knowledge of the exceptions to the SOP and the potential security issues that may arise if not used properly.

Cookie Tossing Vulnerability: Investigate the consequences of cookie tossing aside from session fixation and how they can be utilized.

Cookie Attacks: Security researcher Filedescriptor has conducted thorough research on the nuances of cookies and how they can be exploited by attackers. The presentation is entitled "The Cookie Monster in Your Browser" [*https://hitcon.org/2019/CMT/slide-files/d1_s3_r0.pdf*].

Address Bar Spoofing Vulnerabilities: Explore the different types of address bar spoofing vulnerabilities. For more detailed analysis, you can find numerous write-ups on **rafaybaloch.com**.